分布式开发笔记

分布式理论

什么是分布式系统？

在《分布式系统原理与范型》一书中有如下定义：“分布式系统是若干独立计算机的集合，这些计算机对于用户来说就像单个相关系统”；

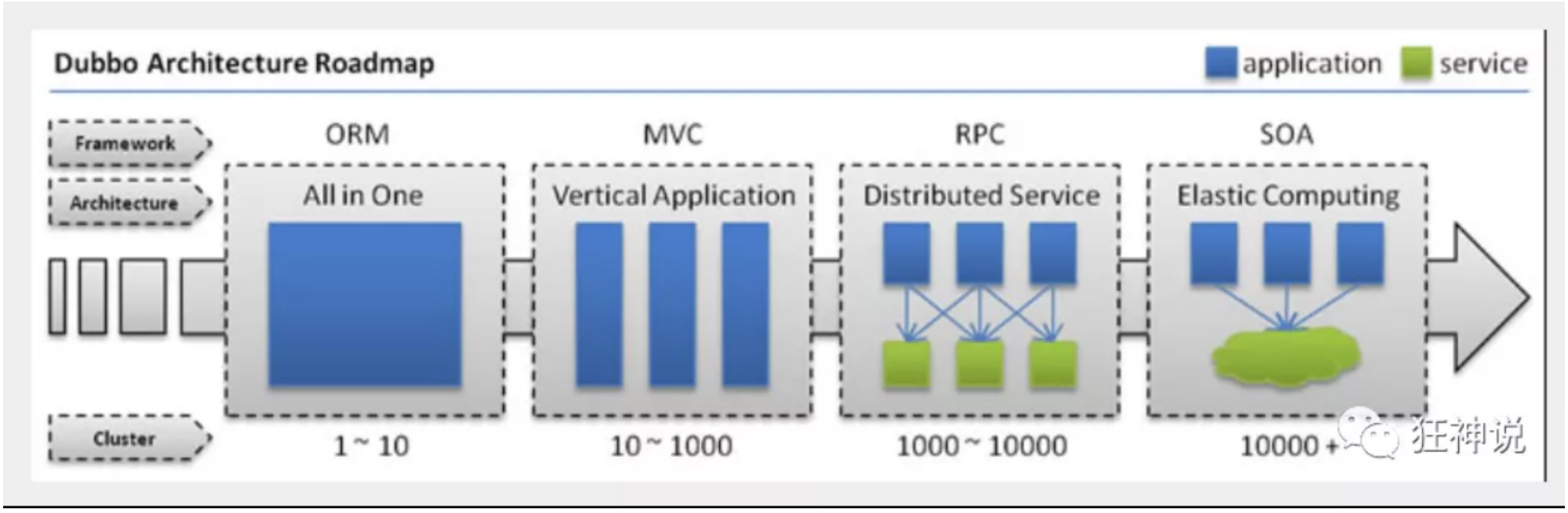
分布式系统是由一组通过网络进行通信、为了完成共同的任务而协调工作的计算机节点组成的系统。分布式系统的出现是为了用廉价的、普通的机器完成单个计算机无法完成的计算、存储任务。其目的是**利用更多的机器，处理更多的数据**。

分布式系统（distributed system）是建立在网络之上的软件系统。

首先需要明确的是，只有当单个节点的处理能力无法满足日益增长的计算、存储任务的时候，且硬件的提升（加内存、加磁盘、使用更好的CPU）高昂到得不偿失的时候，应用程序也不能进一步优化的时候，我们才需要考虑分布式系统。因为，分布式系统要解决的问题本身就是和单机系统一样的，而由于分布式系统多节点、通过网络通信的拓扑结构，会引入很多单机系统没有的问题，为了解决这些问题又会引入更多的机制、协议，带来更多的问题。

Dubbo文档

随着互联网的发展，网站应用的规模不断扩大，常规的垂直应用架构已无法应对，分布式服务架构以及流动计算架构势在必行，急需**一个治理系统**确保架构有条不紊的演进。



单一应用架构

当网站流量很小时，只需一个应用，将所有功能都部署在一起，以减少部署节点和成本。此时，用于简化增删改查工作量的数据访问框架(ORM)是关键。

适用于小型网站，小型管理系统，将所有功能都部署到一个功能里，简单易用。

缺点：

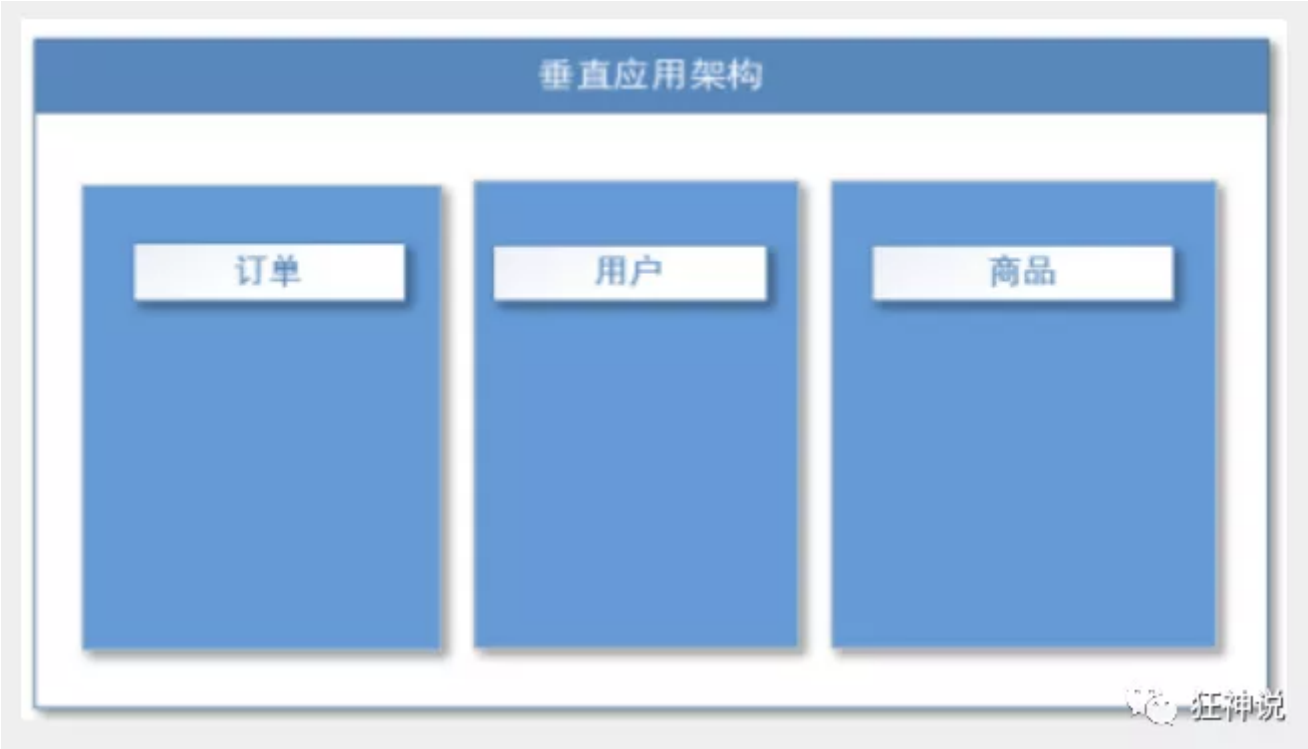
1、性能扩展比较难

2、协同开发问题

3、不利于升级维护

垂直应用架构

当访问量逐渐增大，单一应用增加机器带来的加速度越来越小，将应用拆成互不相干的几个应用，以提升效率。此时，用于加速前端页面开发的Web框架(MVC)是关键。

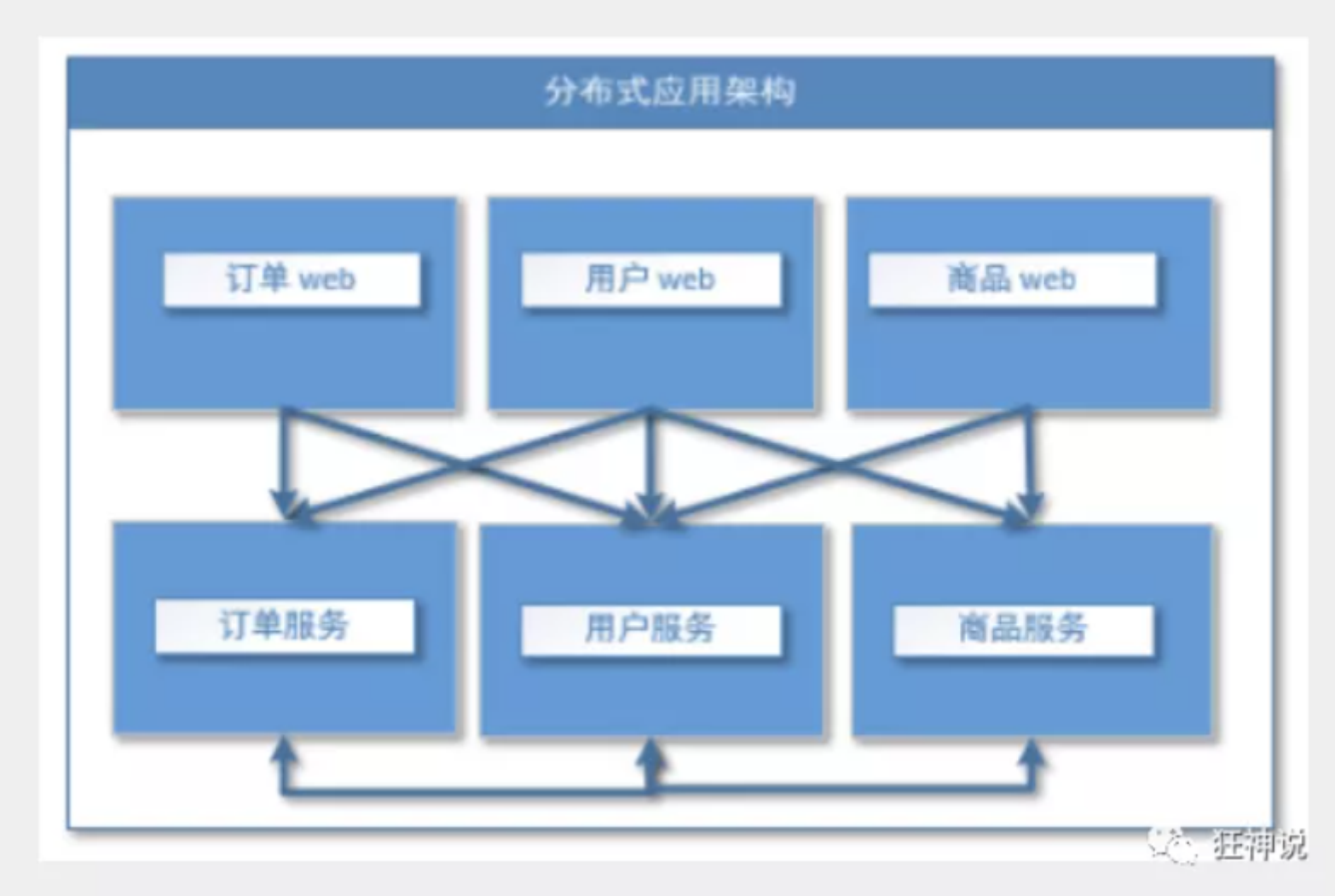


通过切分业务来实现各个模块独立部署，降低了维护和部署的难度，团队各司其职更易管理，性能扩展也更方便，更有针对性。

缺点：公用模块无法重复利用，开发性的浪费

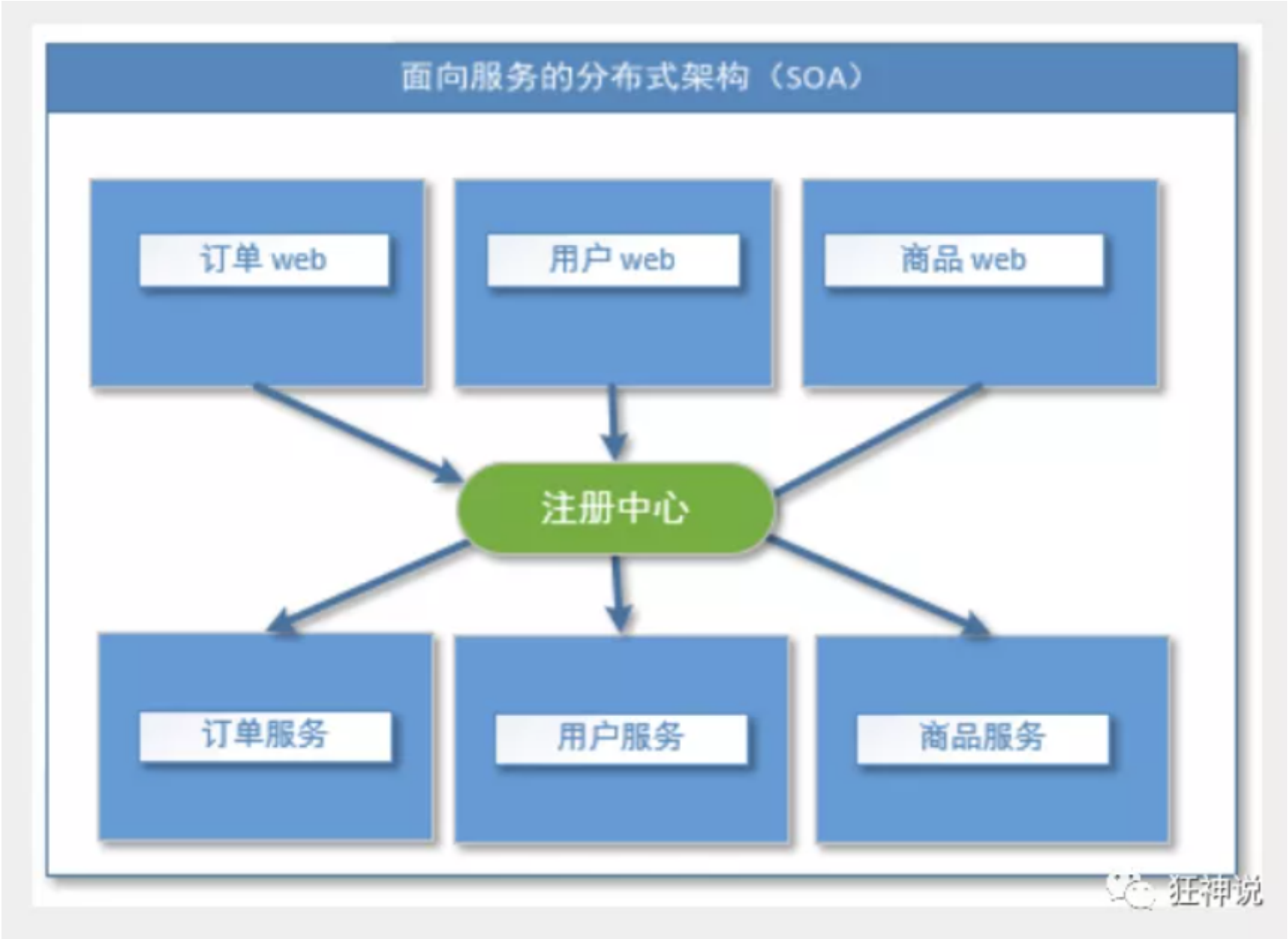
分布式服务架构

当垂直应用越来越多，应用之间交互不可避免，将核心业务抽取出来，作为独立的服务，逐渐形成稳定的服务中心，使前端应用能更快速的响应多变的市场需求。此时，用于提高业务复用及整合的 **分布式服务框架(RPC)** 是关键。



流动计算架构

当服务越来越多，容量的评估，小服务资源的浪费等问题逐渐显现，此时需增加一个调度中心基于访问压力实时管理集群容量，提高集群利用率。此时，用于 **提高机器利用率的资源调度和治理中心** (SOA)[Service Oriented Architecture]是关键。



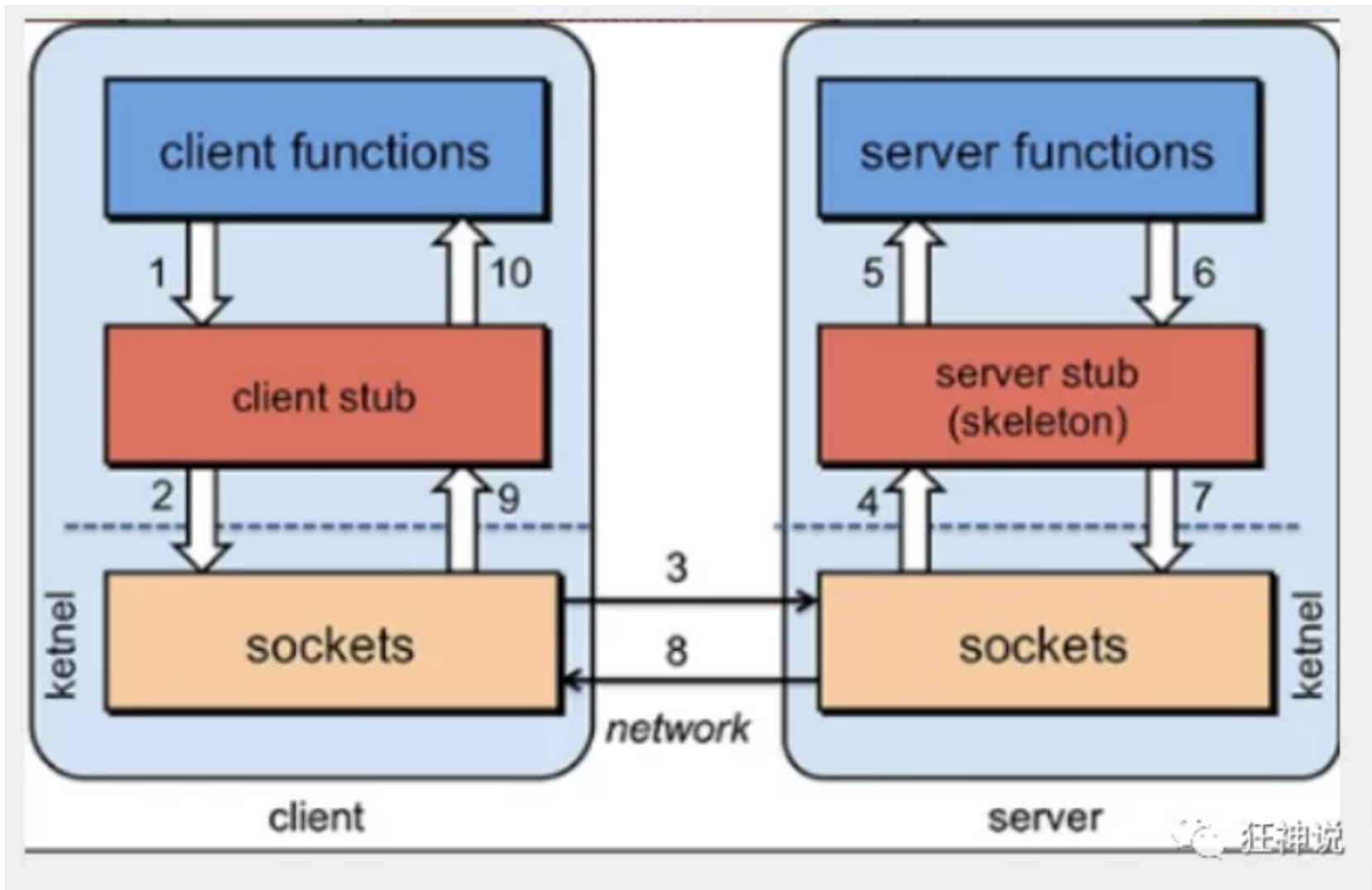
什么是RPC?

RPC【Remote Procedure Call】是指远程过程调用，是一种进程间通信方式，他是一种技术的思想，而不是规范。它允许程序调用另一个地址空间（通常是共享网络的另一台机器上）的过程或函数，而不用程序员显式编码这个远程调用的细节。即程序员无论是调用本地的还是远程的函数，本质上编写的调用代码基本相同。

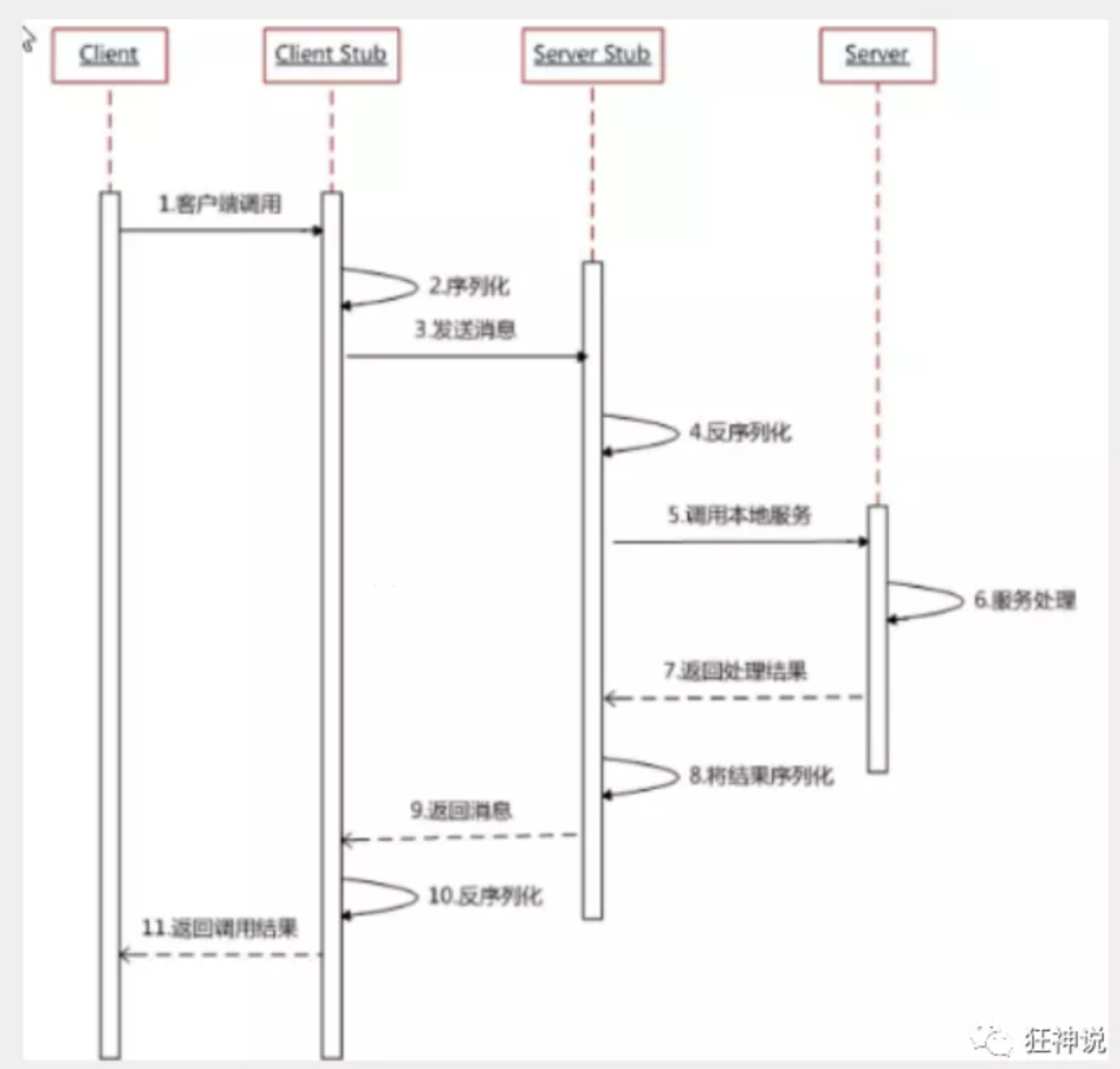
也就是说两台服务器A，B，一个应用部署在A服务器上，想要调用B服务器上应用提供的函数/方法，由于不在一个内存空间，不能直接调用，需要通过网络来表达调用的语义和传达调用的数据。为什么要用RPC呢？就是无法在一个进程内，甚至一个计算机内通过本地调用的方式完成的需求，比如不同的系统间的通讯，甚至不同的组织间的通讯，由于计算能力需要横向扩展，需要在多台机器组成的集群上部署应用。RPC就是要像调用本地的函数一样去调远程函数；

推荐阅读文章：https://www.jianshu.com/p/2accc2840a1b

RPC基本原理



步骤解析：



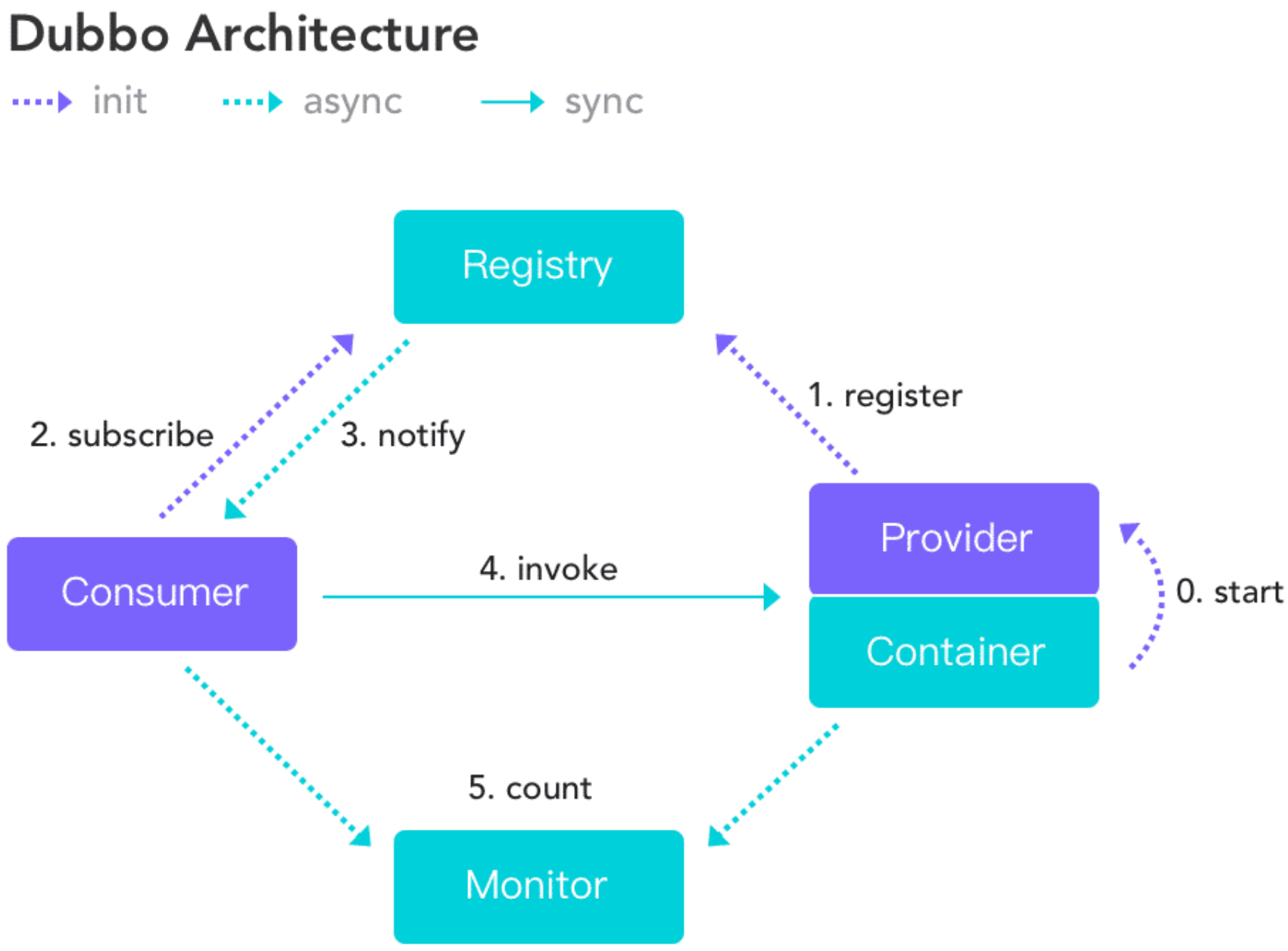
RPC两个核心模块：通讯，序列化。

Dubbo

概念

官方文档：https://dubbo.apache.org/zh/docs/

1. Dubbo特性：https://dubbo.apache.org/zh/docs/new-in-dubbo3/
2. **dubbo基本概念**



服务提供者（Provider）：暴露服务的服务提供方，服务提供者在启动时，向注册中心注册自己提供的服务。

服务消费者（Consumer）：调用远程服务的服务消费方，服务消费者在启动时，向注册中心订阅自己所需的服务，服务消费者，从提供者地址列表中，基于软负载均衡算法，选一台提供者进行调用，如果调用失败，再选另一台调用。

注册中心（Registry）：注册中心返回服务提供者地址列表给消费者，如果有变更，注册中心将基于长连接推送变更数据给消费者

监控中心（Monitor）：服务消费者和提供者，在内存中累计调用次数和调用时间，定时每分钟发送一次统计数据到监控中心

调用关系说明

l 服务容器负责启动，加载，运行服务提供者。

l 服务提供者在启动时，向注册中心注册自己提供的服务。

l 服务消费者在启动时，向注册中心订阅自己所需的服务。

l 注册中心返回服务提供者地址列表给消费者，如果有变更，注册中心将基于长连接推送变更数据给消费者。

l 服务消费者，从提供者地址列表中，基于软负载均衡算法，选一台提供者进行调用，如果调用失败，再选另一台调用。

l 服务消费者和提供者，在内存中累计调用次数和调用时间，定时每分钟发送一次统计数据到监控中心。

安装

相应安装包

链接：https://pan.baidu.com/s/1fqJnjAPrqbUBcx1rTZBaMQ
提取码：uyuo

JDK版本

一定要为jdk1.8

安装zookeeper

点进dubbo官方文档，推荐我们使用Zookeeper 注册中心

什么是zookeeper呢？可以查看官方文档

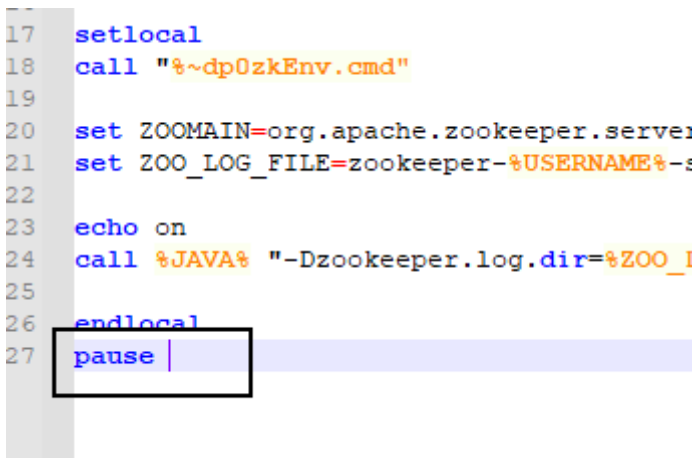
- 下载zookeeper：https://archive.apache.org/dist/zookeeper

- 解压zookeeper

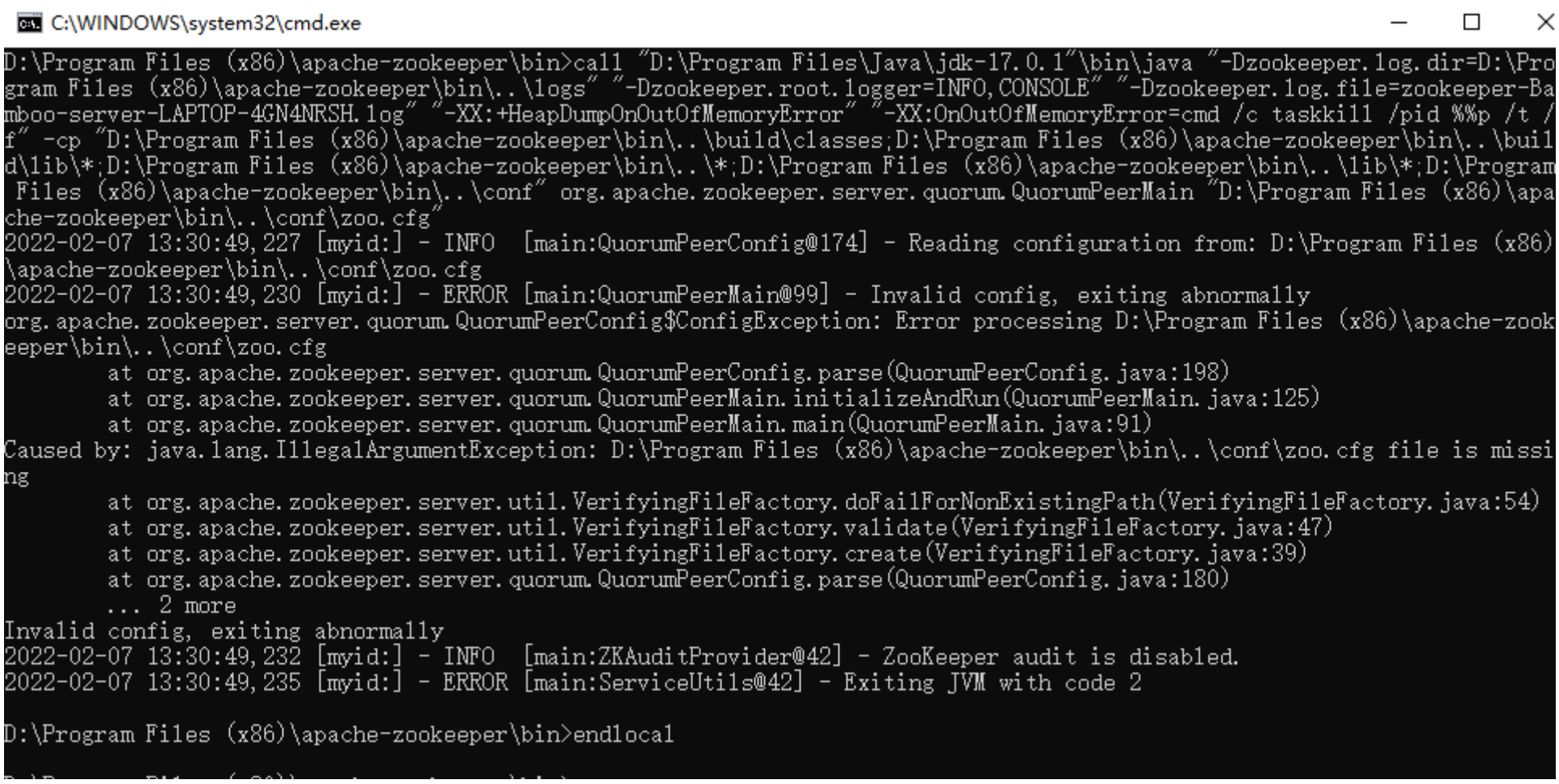
- 运行 `/bin/zkServer.cmd` ，初次运行会报错，没有zoo.cfg配置文件；

可能遇到问题：闪退！

解决方案：编辑zkServer.cmd文件末尾添加pause 。这样运行出错就不会退出，会提示错误信息，方便找到原因。



继续运行



- 修改zoo.cfg配置文件

将conf文件夹下面的zoo_sample.cfg复制一份改名为zoo.cfg即可。

注意几个重要位置：

`dataDir=./` 临时数据存储的目录（可写相对路径）

`clientPort=2181` zookeeper的端口号

修改完成后再次启动zookeeper，之后不要关闭

```
C:\WINDOWS\system32\cmd.exe
2022-02-07 13:34:21,958 [myid:] - INFO  [main:DefaultSessionIdManager@339] - No SessionScavenger set, using defaults
2022-02-07 13:34:21,959 [myid:] - INFO  [main:HouseKeeper@132] - node0 Scavenging every 660000ms
2022-02-07 13:34:21,961 [myid:] - WARN  [main:ConstraintSecurityHandler@759] - ServletContext@o.e.j.s.ServletContextHandler@3febb011{/,null,STARTING} has unc
overed http methods for path: /*
2022-02-07 13:34:21,967 [myid:] - INFO  [main:ContextHandler@916] - Started o.e.j.s.ServletContextHandler@3febb011{/,null,AVAILABLE}
2022-02-07 13:34:22,099 [myid:] - INFO  [main:AbstractConnector@331] - Started ServerConnector@7a69b07{HTTP/1.1, (http/1.1)}{0.0.0.0:8080}
2022-02-07 13:34:22,099 [myid:] - INFO  [main:Server@415] - Started @592ms
2022-02-07 13:34:22,100 [myid:] - INFO  [main:JettyAdminServer@181] - Started AdminServer on address 0.0.0.0, port 8080 and command URL /commands
2022-02-07 13:34:22,103 [myid:] - INFO  [main:ServerCnxnFactory@169] - Using org.apache.zookeeper.server.NIOServerCnxnFactory as server connection factory
2022-02-07 13:34:22,104 [myid:] - WARN  [main:ServerCnxnFactory@309] - maxCnxns is not configured, using default value 0.
2022-02-07 13:34:22,105 [myid:] - INFO  [main:NIOServerCnxnFactory@652] - Configuring NIO connection handler with 10s sessionless connection timeout, 2 selec
tor thread(s), 32 worker threads, and 64 kB direct buffers.
2022-02-07 13:34:22,111 [myid:] - INFO  [main:NIOServerCnxnFactory@660] - binding to port 0.0.0.0/0.0.0.0:2181
2022-02-07 13:34:22,121 [myid:] - INFO  [main:WatchManagerFactory@42] - Using org.apache.zookeeper.server.watch.WatchManager as watch manager
2022-02-07 13:34:22,121 [myid:] - INFO  [main:WatchManagerFactory@42] - Using org.apache.zookeeper.server.watch.WatchManager as watch manager
2022-02-07 13:34:22,122 [myid:] - INFO  [main:ZKDatabase@133] - zookeeper.snapshotSizeFactor = 0.33
2022-02-07 13:34:22,122 [myid:] - INFO  [main:ZKDatabase@153] - zookeeper.commitLogCount=500
2022-02-07 13:34:22,128 [myid:] - INFO  [main:SnapStream@61] - zookeeper.snapshot.compression.method = CHECKED
2022-02-07 13:34:22,129 [myid:] - INFO  [main:FileTxnSnapLog@479] - Snapshotting: 0x0 to \tmp\zookeeper\version-2\snapshot.0
2022-02-07 13:34:22,133 [myid:] - INFO  [main:ZKDatabase@290] - Snapshot loaded in 10 ms, highest zxid is 0x0, digest is 1371985504
2022-02-07 13:34:22,134 [myid:] - INFO  [main:FileTxnSnapLog@479] - Snapshotting: 0x0 to \tmp\zookeeper\version-2\snapshot.0
2022-02-07 13:34:22,135 [myid:] - INFO  [main:ZooKeeperServer@543] - Snapshot taken in 1 ms
2022-02-07 13:34:22,141 [myid:] - INFO  [main:RequestThrottler@75] - zookeeper.request_throttler.shutdownTimeout = 10000
2022-02-07 13:34:22,141 [myid:] - INFO  [ProcessThread(sid:0 cport:2181)::PrepRequestProcessor@137] - PrepRequestProcessor (sid:0) started, reconfigEnabled=f
alse
2022-02-07 13:34:22,153 [myid:] - INFO  [main:ContainerManager@84] - Using checkIntervalMs=60000 maxPerMinute=10000 maxNeverUsedIntervalMs=0
2022-02-07 13:34:22,154 [myid:] - INFO  [main:ZKAuditProvider@42] - ZooKeeper audit is disabled.
```

- 使用zkCli.cmd测试

ls /：列出zookeeper根下保存的所有节点

```
WatchedEvent state:SyncConnected type:None path:null
[zk: localhost:2181(CONNECTED) 0] ls /
[zookeeper]
[zk: localhost:2181(CONNECTED) 1]
```

create –e /hutao 123：创建一个hutao节点，值为123

```
[zk: localhost:2181(CONNECTED) 1] create -e /hutao 123
Created /hutao
[zk: localhost:2181(CONNECTED) 2]
```

get /hutao ： 获取/hutao 节点的值

```
[zk: localhost:2181(CONNECTED) 2] get /hutao
123
[zk: localhost:2181(CONNECTED) 3]
```

再来查看一下节点

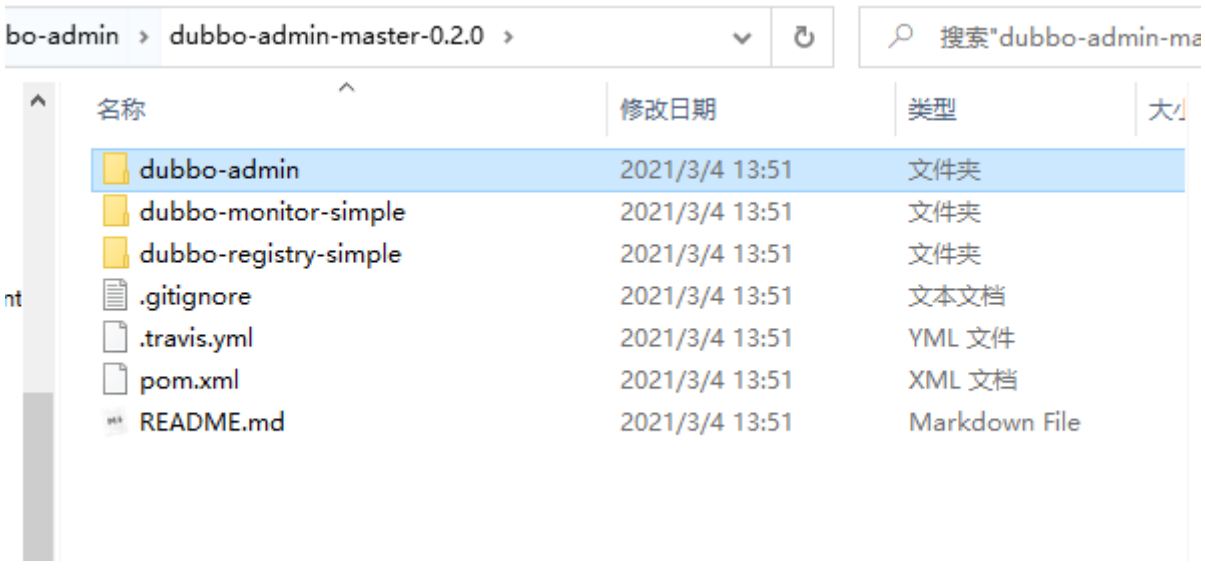
```
[zk: localhost:2181(CONNECTED) 3] ls /
[hutao, zookeeper]
[zk: localhost:2181(CONNECTED) 4]
```

安装dubbo-admin

dubbo本身并不是一个服务软件。它其实就是一个jar包，能够帮你的java程序连接到zookeeper，并利用zookeeper消费、提供服务。

但是为了让用户更好的管理监控众多的dubbo服务，官方提供了一个可视化的监控程序dubbo-admin，不过这个监控即使不装也不影响使用。

1. **下载dubbo-admin**： https://github.com/apache/dubbo-admin/tree/master-0.2.0
2. **解压进入目录**



修改 `dubbo-admin-server\src\main\resources\application.properties` 指定zookeeper地址

```
server.port=7001
spring.velocity.cache=false
spring.velocity.charset=UTF-8
spring.velocity.layout-url=/templates/default.vm
spring.messages.fallback-to-system-locale=false
spring.messages.basename=i18n/message
spring.root.password=root
spring.guest.password=guest

dubbo.registry.address=zookeeper://127.0.0.1:2181
```

3. 配置Apache Maven：https://maven.apache.org/download.cgi

- 变量名：MAVEN_HOME
- 变量值：E:\apache-maven-3.2.3
- 变量名：Path
- 变量值：;%MVN_HOME%\bin
- 打开cmd输入 `echo %MAVEN_HOME%` ，测试是否安装成功

4. **在项目目录下**打包dubbo-admin

```
mvn clean package -Dmaven.test.skip=true
```

```
[INFO] Not compiling test sources
[INFO]
[INFO] --- maven-surefire-plugin:2.21.0:test (default-test) @ dubbo-admin-test ---
[INFO] Tests are skipped.
[INFO]
[INFO] --- jacoco-maven-plugin:0.8.2:report (report) @ dubbo-admin-test ---
[INFO] Skipping JaCoCo execution due to missing execution data file.
[INFO]
[INFO] --- maven-jar-plugin:3.0.2:jar (default-jar) @ dubbo-admin-test ---
[INFO] Building jar: D:\Program Files (x86)\apache-dubbo-admin\dubbo-admin-master\dubbo-admin-test\target\dubbo-admin-te
st-0.3.0.jar
[INFO]
[INFO] --- maven-site-plugin:3.7:attach-descriptor (attach-descriptor) @ dubbo-admin-test ---
[INFO] Skipping because packaging 'jar' is not pom.
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for dubbo-admin 0.3.0:
[INFO]
[INFO] dubbo-admin ........................................ SUCCESS [01:58 min]
[INFO] dubbo-admin-ui ..................................... SUCCESS [ 50.533 s]
[INFO] dubbo-admin-server ................................. SUCCESS [02:24 min]
[INFO] dubbo-admin-distribution ........................... SUCCESS [  1.640 s]
[INFO] dubbo-admin-test ................................... SUCCESS [01:07 min]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  06:22 min
[INFO] Finished at: 2022-02-07T15:12:20+08:00
[INFO] ------------------------------------------------------------------------

D:\Program Files (x86)\apache-dubbo-admin\dubbo-admin-master>
```

5. 执行 `dubbo-admin\target` 下的 `dubbo-admin-0.0.1-SNAPSHOT.jar`

```
java -jar dubbo-admin-0.0.1-SNAPSHOT.jar
```

【注意：zookeeper的服务一定要打开！】

执行完毕，去访问一下 http://localhost:7001/ ， 这时候我们需要输入登录账户和密码，我们都是默认的root-root；

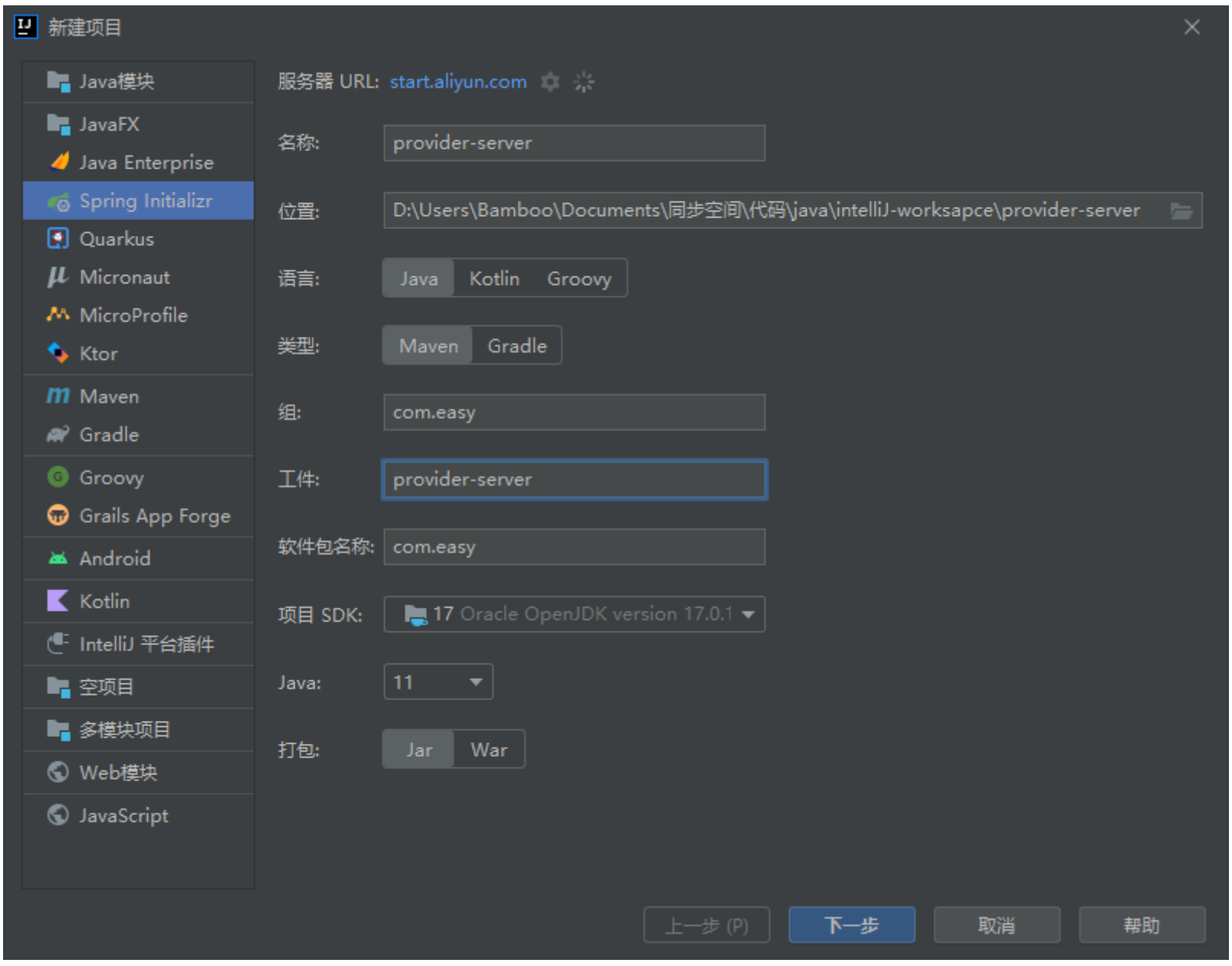
登录成功后，查看界面

或者通过idea打开项目，并且将java版本修改为1.8，并查看pom.xml，然后运行 `dubbo-admin` 即可。

服务注册分发(SpringBoot + Dubbo + zookeeper)

准备工作

- 启动zookeeper
- 启动dubbo-admin
- 使用IDEA创建一个Maven项目 `dubbo_zookeeper_demo`
- 创建一个模块 `provider-server` ，选择web依赖即可



- **项目创建完毕，写一个服务，比如卖票的服务；**

编写接口 `service/TicketService.java`

```
package com.easy.service;

public interface TicketService {
    public String getTicket();
}

```

编写实现类

```
package com.easy.service;

public class TicketServiceImpl implements TicketService{
    @Override
    public String getTicket(){
        return "Dubbo+Zookeeper+SpringBoot实战教程";
    }
}
```

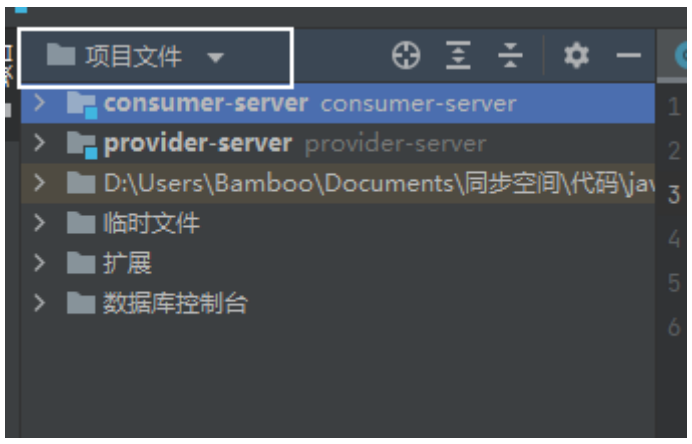
- **创建一个模块，实现服务消费者：consumer-server ， 选择web依赖即可**
- **项目创建完毕，写一个服务，比如用户的服务；**

编写service

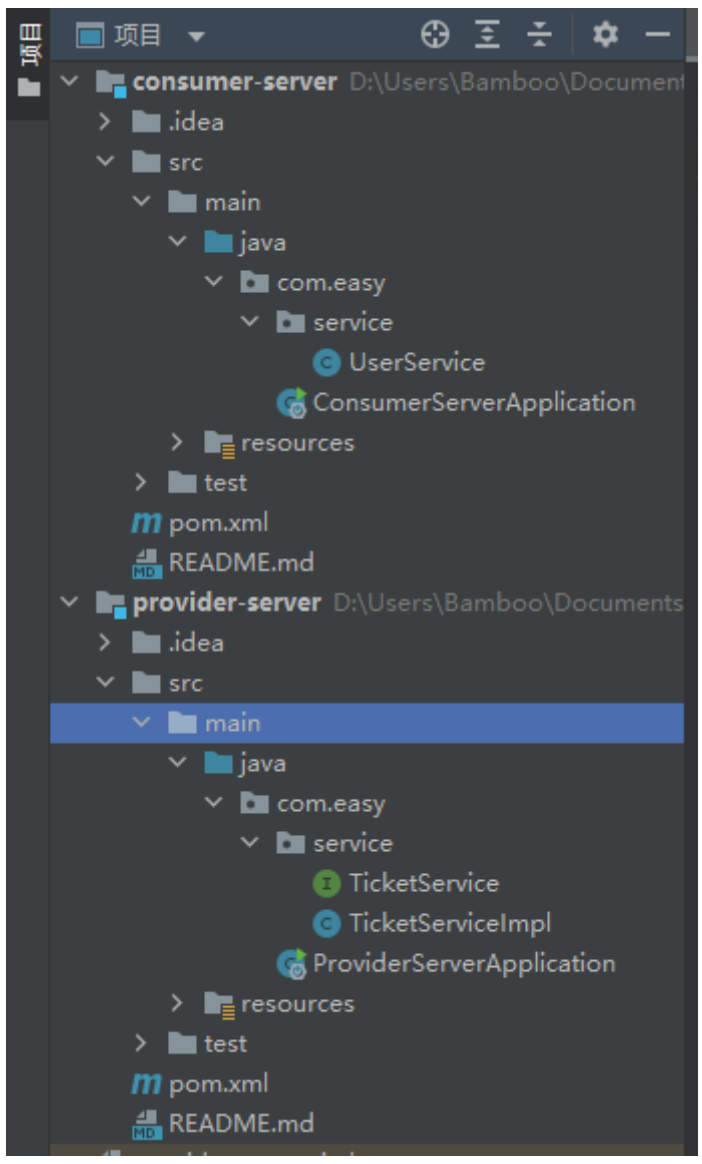
```
package com.easy.service;

public class UserService {
    // 想拿到provider-server提供的票
}
```

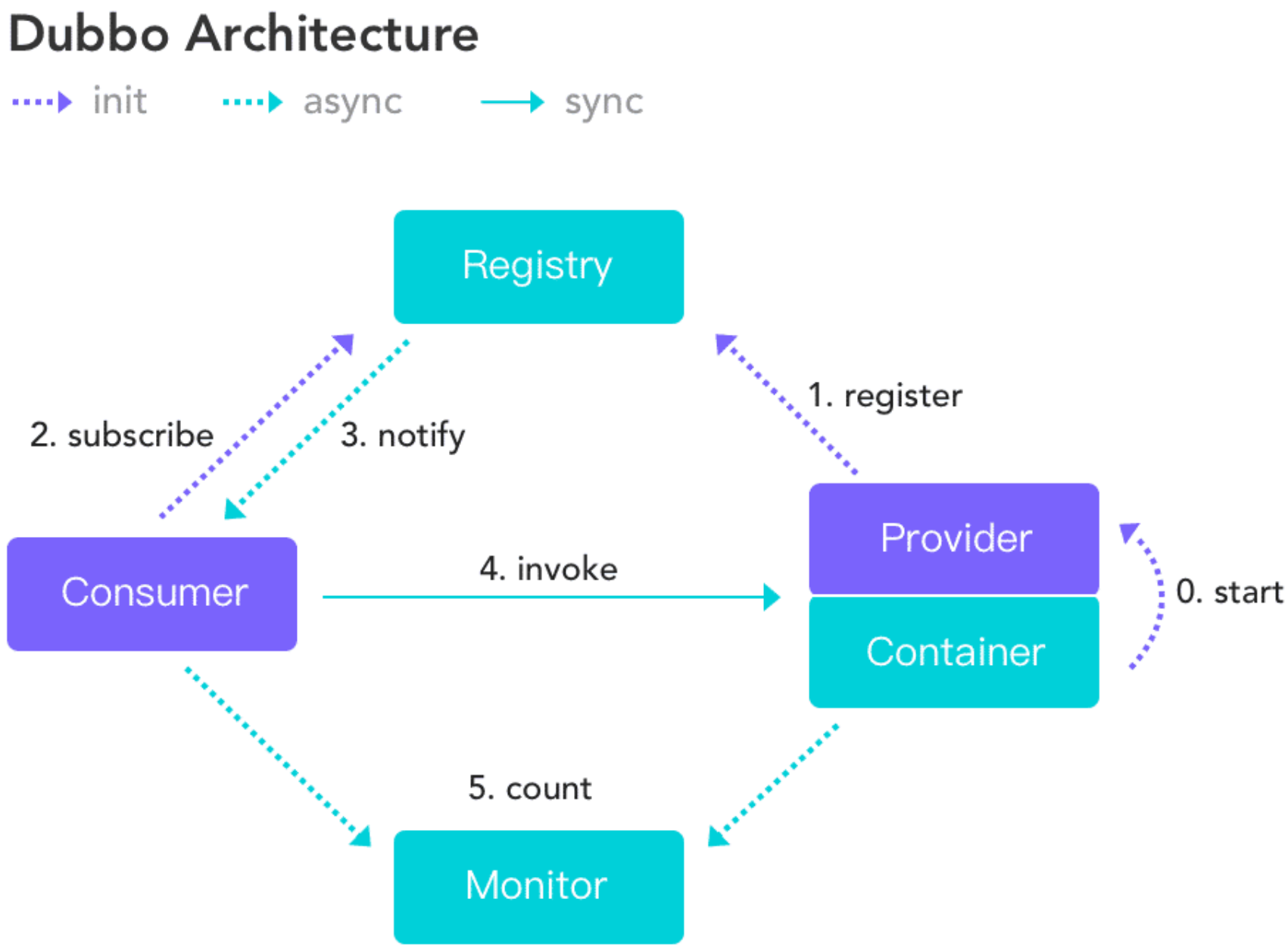
- 如果想让两个文件夹同时出现在 `dubbo-zookeeper-demo` 文件夹下的话，先切换到项目文件，先显示所有文件夹。



然后在每个工程文件下找到文件pom.xml，如果没有，则新建一个并填写好内容。并在pom.xml 文件上右键 Add as Maven Project，这样就将文件夹变成了一个maven项目。这样就可以同时存在了。之后切换回项目。



需求：现在用户想使用买票的服务，这要怎么弄呢？



服务提供者

- **将服务提供者注册到注册中心，我们需要整合Dubbo和zookeeper，所以需要导包**
  从dubbo官网进入github，看下方的帮助文档，找到dubbo-springboot，找到依赖包

```
<!-- Dubbo Spring Boot Starter -->
<!-- https://mvnrepository.com/artifact/org.apache.dubbo/dubbo-spring-boot-starter -->
<dependency>
    <groupId>org.apache.dubbo</groupId>
    <artifactId>dubbo-spring-boot-starter</artifactId>
    <version>2.7.3</version>
</dependency>

```

zookeeper去maven仓库下载，zkclient；

```
<!-- https://mvnrepository.com/artifact/com.github.sgroschupf/zkclient -->
<dependency>
   <groupId>com.github.sgroschupf</groupId>
   <artifactId>zkclient</artifactId>
   <version>0.1</version>
</dependency>
```

【新版的坑】zookeeper及其依赖包，解决日志冲突，还需要剔除日志依赖；

```
<!-- 引入zookeeper -->
<dependency>
   <groupId>org.apache.curator</groupId>
   <artifactId>curator-framework</artifactId>
   <version>2.12.0</version>
</dependency>
<dependency>
   <groupId>org.apache.curator</groupId>
   <artifactId>curator-recipes</artifactId>
   <version>2.12.0</version>
</dependency>
```

```
<dependency>
   <groupId>org.apache.zookeeper</groupId>
   <artifactId>zookeeper</artifactId>
   <version>3.4.14</version>
   <!--排除这个slf4j-log4j12-->
   <exclusions>
       <exclusion>
           <groupId>org.slf4j</groupId>
           <artifactId>slf4j-log4j12</artifactId>
       </exclusion>
   </exclusions>
</dependency>
```

- 配置参数

```
#当前应用名字
dubbo.application.name=provider-server
#注册中心地址
dubbo.registry.address=zookeeper://127.0.0.1:2181
#扫描指定包下服务/那些服务要被注册
dubbo.scan.base-packages=com.easy.service
```

- 在service的实现类 `TicketServiceImpl` 中配置服务注解，发布服务！注意导包问题**

```
package com.easy.service;

// zookeeper：服务注册与发现

import org.apache.dubbo.config.annotation.Service;
import org.springframework.stereotype.Component;

@Service  // 可以被扫描到，在项目启动后就会被自动注册到注册中心
@Component // 使用了Dubbo后尽量不要使用Service
public class TicketServiceImpl implements TicketService{
    @Override
    public String getTicket(){
        return "Dubbo+Zookeeper+SpringBoot实战教程";
    }
}

```

逻辑理解 ： 应用启动起来，dubbo就会扫描指定的包下带有@component注解的服务，将它发布在指定的注册中心中！

- 启动程序

服务消费者

现在已经部署好了服务，消费者应用需要进行调用该业务，如何使用？

- 导入依赖，和之前的依赖一样；

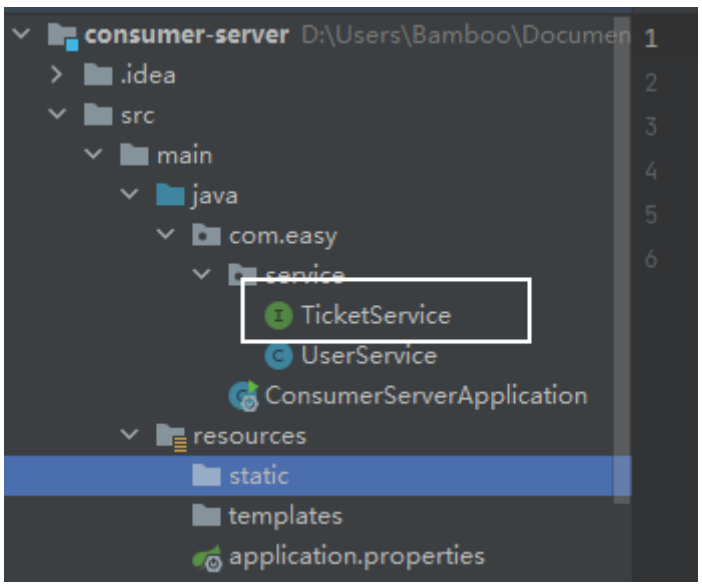
```
<!-- Dubbo Spring Boot Starter -->
<!-- https://mvnrepository.com/artifact/org.apache.dubbo/dubbo-spring-boot-starter -->
<dependency>
    <groupId>org.apache.dubbo</groupId>
    <artifactId>dubbo-spring-boot-starter</artifactId>
    <version>2.7.3</version>
</dependency>
<!-- https://mvnrepository.com/artifact/com.github.sgroschupf/zkclient -->
<dependency>
   <groupId>com.github.sgroschupf</groupId>
   <artifactId>zkclient</artifactId>
   <version>0.1</version>
</dependency>
```

```
<!-- 引入zookeeper -->
<dependency>
   <groupId>org.apache.curator</groupId>
   <artifactId>curator-framework</artifactId>
   <version>2.12.0</version>
</dependency>
<dependency>
   <groupId>org.apache.curator</groupId>
   <artifactId>curator-recipes</artifactId>
   <version>2.12.0</version>
</dependency>
<dependency>
   <groupId>org.apache.zookeeper</groupId>
   <artifactId>zookeeper</artifactId>
   <version>3.4.14</version>
   <!--排除这个slf4j-log4j12-->
   <exclusions>
       <exclusion>
           <groupId>org.slf4j</groupId>
           <artifactId>slf4j-log4j12</artifactId>
       </exclusion>
   </exclusions>
</dependency>
```

- **配置参数**

```
#当前应用名字/消费者去哪里拿服务，需要暴露自己的名字
dubbo.application.name=consumer-server
#注册中心地址
dubbo.registry.address=zookeeper://127.0.0.1:2181
```

- **本来正常步骤是需要将服务提供者的接口打包，然后用pom文件导入，这里使用简单的方式，直接将服务的接口拿过来，路径必须保证正确，即和服务提供者相同；**



> 只拿接口

- **完善消费者的服务类**

```java
package com.easy.service;

import org.apache.dubbo.config.annotation.Reference;
import org.apache.dubbo.config.annotation.Service;
import org.springframework.stereotype.Component;

@Service
@Component
public class UserService {

    // 想拿到provider-server提供的票，要去注册中心拿到服务
    @Reference // 两种方式：1.pom坐标 2.定义相同路径的接口方法，因为具体实现是在其他服务器上进行逻辑运算的
    TicketService ticketService;

    public void buyTicket(){
       String ticket = ticketService.getTicket();
       System.out.println("在注册中心拿到=>"+ticket);
    }
}

```

- **测试类编写；**

```java
@SpringBootTest
class ConsumerServerApplicationTests {

    @Autowired
    UserService userService;

    @Test
    void contextLoads() {
        userService.buyTicket();
    }

}
```

启动测试

1. 开启zookeeper

2. 打开dubbo-admin实现监控【可以不用做】

3. 开启服务者

4. 消费者消费测试

结果：

```
2022-02-08 03:21:10.386  INFO 30900 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor
2022-02-08 03:21:10.643  INFO 30900 --- [           main] com.easy.ConsumerServerApplicationTests
在注册中心拿到=>Dubbo+Zookeeper+SpringBoot实战教程
2022-02-08 03:21:10.940  INFO 30900 --- [extShutdownHook] .b.c.e.AwaitingNonWebApplicationListener
2022-02-08 03:21:10.945  INFO 30900 --- [tor-Framework-0] o.a.c.f.imps.CuratorFrameworkImpl
2022-02-08 03:21:10.948  INFO 30900 --- [extShutdownHook] org.apache.zookeeper.ZooKeeper
2022-02-08 03:21:10.948  INFO 30900 --- [ain-EventThread] org.apache.zookeeper.ClientCnxn
```

监控中心：

这就是SpingBoot + dubbo + zookeeper实现分布式开发的应用，其实就是一个服务拆分的思想；